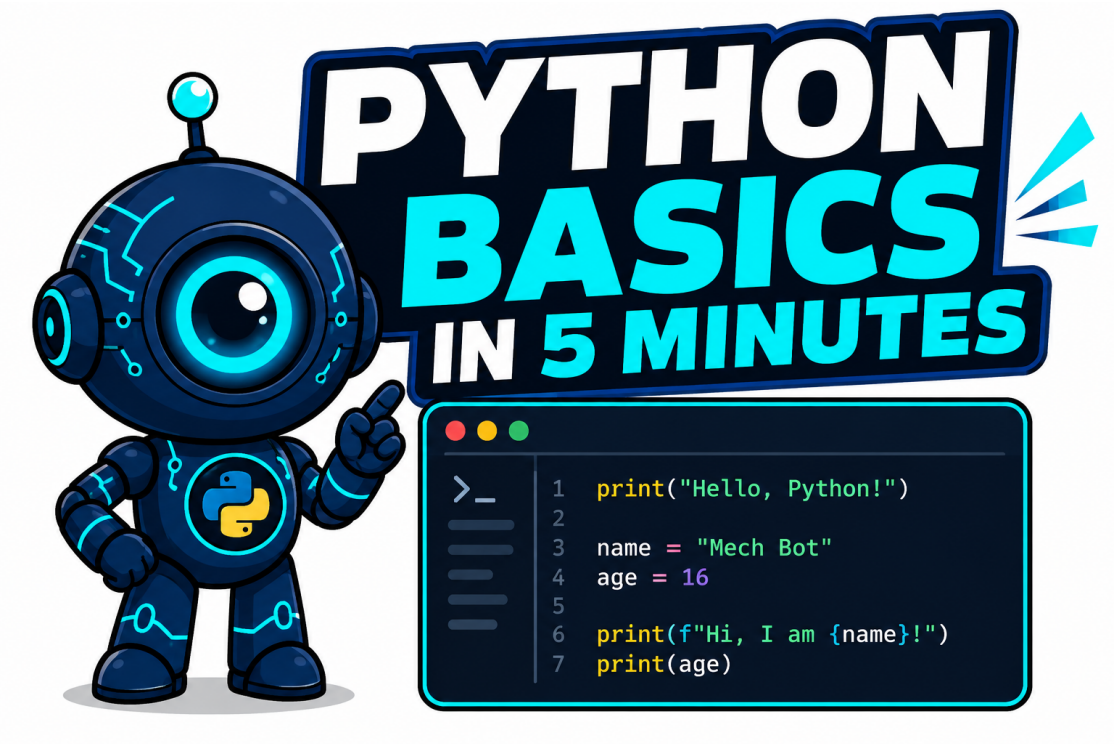
AI MECH LAB

AI Course — Beginner Track



Module 2, Topic 2.1

Python Essentials

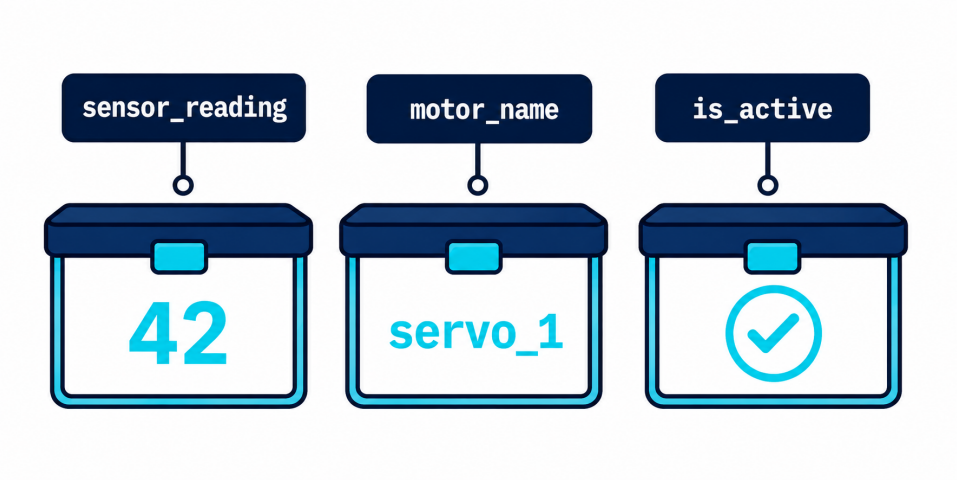
Lesson

Every AI system you'll learn about in this course is built with code, and the language you'll use throughout this platform is Python — it's the standard language for AI and machine learning work, for reasons you'll appreciate more once you start using libraries built specifically for handling data and building models. Before touching any of that, you need the basic building blocks: variables, loops, and functions. These three concepts show up in every single script you'll ever write, so this lesson is worth taking slowly.

Variables — storing information

A variable is a name that holds a piece of information so you can use it later without retyping it. In Python, creating a variable is as simple as writing a name, an equals sign, and a value:

```
sensor_reading = 42
motor_name = "servo_1"
is_active = True
```

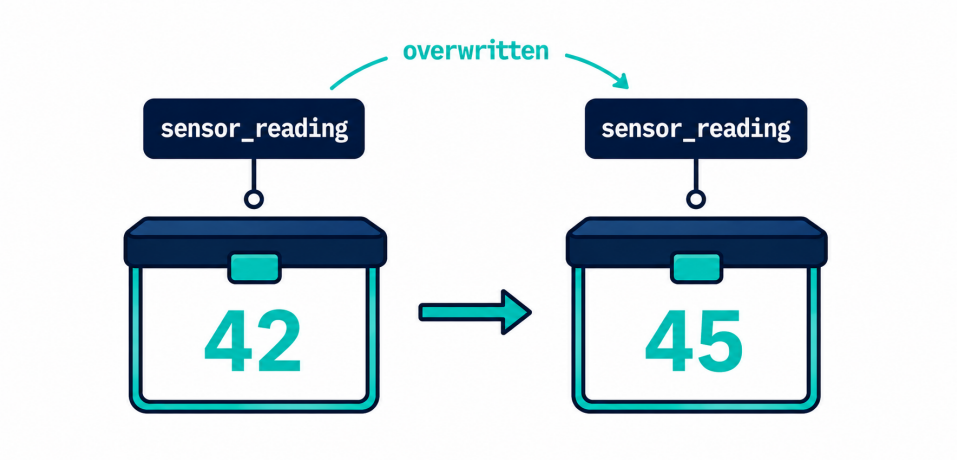


Three variables holding three different types of data.

Notice you didn't have to declare what type of information each variable holds — Python figures that out automatically from the value you give it. sensor_reading is a number (an integer), motor_name is text (called a "string," always wrapped in quotes), and is_active is a true/false value (called a "boolean"). These three types — numbers, strings, and booleans — cover the vast majority of what you'll work with early on.

Once a variable is created, you can use it anywhere in your code, and you can change its value at any time:

```
sensor_reading = 42
sensor_reading = 45
print(sensor_reading)
```



The second assignment overwrites the first — variables are labeled storage you can update.

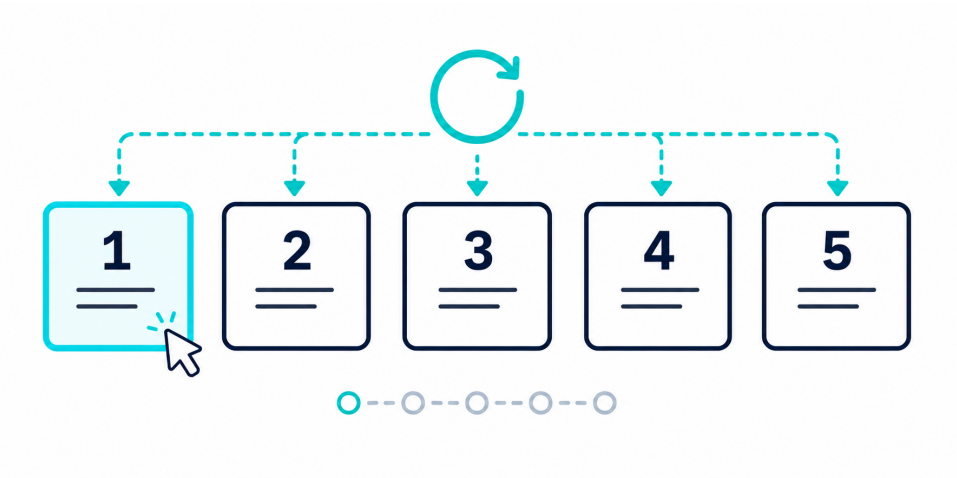
This would print 45 — the second assignment overwrote the first. Variables aren't fixed; they're just labeled storage you can update.

Loops — repeating actions without repeating code

A huge amount of programming is doing the same action many times — checking every value in a list, processing every row in a dataset, running a calculation 1,000 times to train a model. Writing that out manually would be absurd. Loops solve this.

The most common loop in Python is the for loop, which repeats an action once for every item in a collection:

```
sensor_readings = [42, 45, 41, 50, 47]
for reading in sensor_readings:
    print(reading)
```

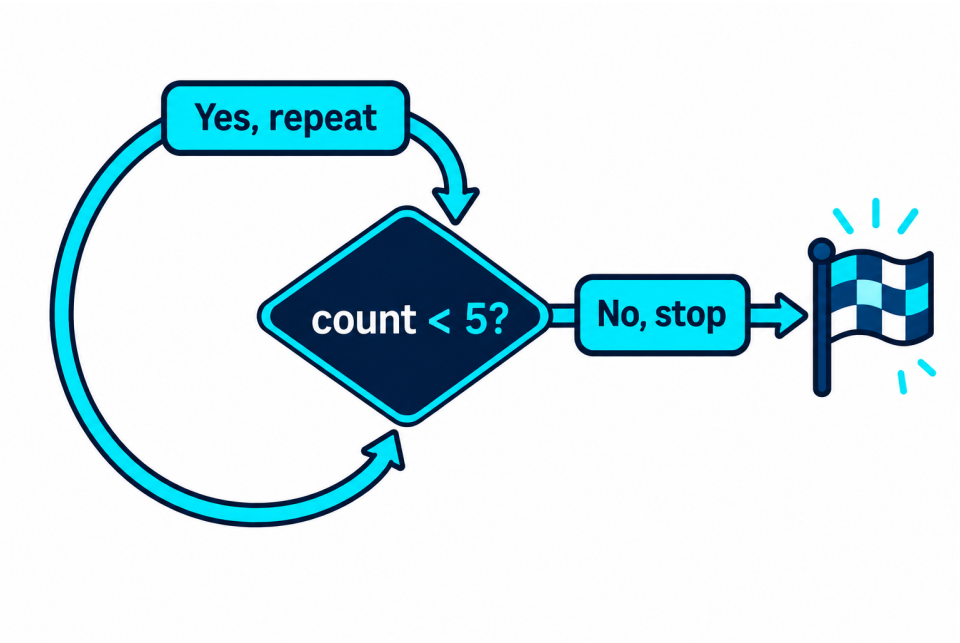


A for loop moving through each item in a collection, one at a time.

This prints each number in the list, one at a time, without you writing five separate print statements. reading is a temporary variable that takes on each value in the list, one at a time, as the loop runs.

The other common loop is the while loop, which repeats as long as a condition stays true:

```
count = 0
while count < 5:
    print(count)
    count = count + 1
```



A while loop keeps repeating until its condition becomes false.

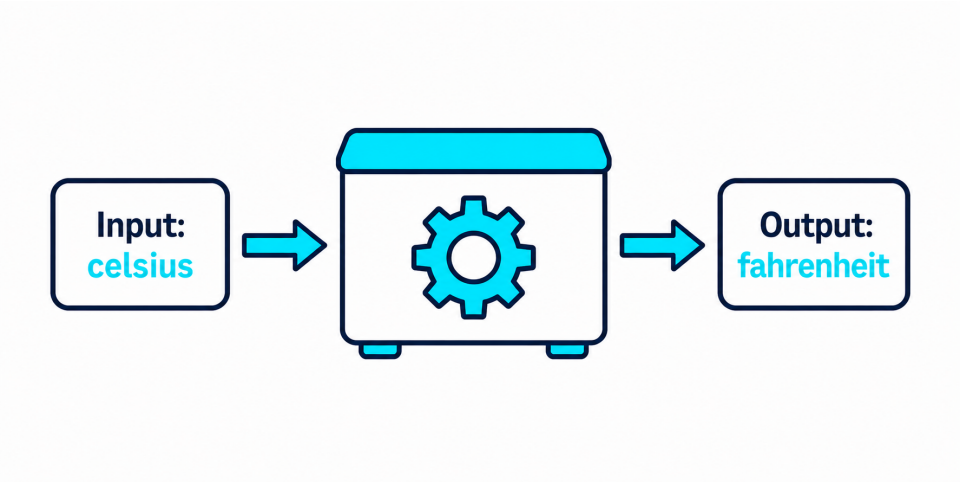
This keeps printing and increasing count until it reaches 5, at which point the condition count < 5 becomes false and the loop stops. while loops are useful when you don't know in advance exactly how many times you'll need to repeat something — you just know the condition that should stop it.

Functions — packaging up reusable logic

A function is a named, reusable block of code that performs a specific task. Instead of writing the same calculation over and over throughout a script, you write it once as a function, then call that function by name whenever you need it.

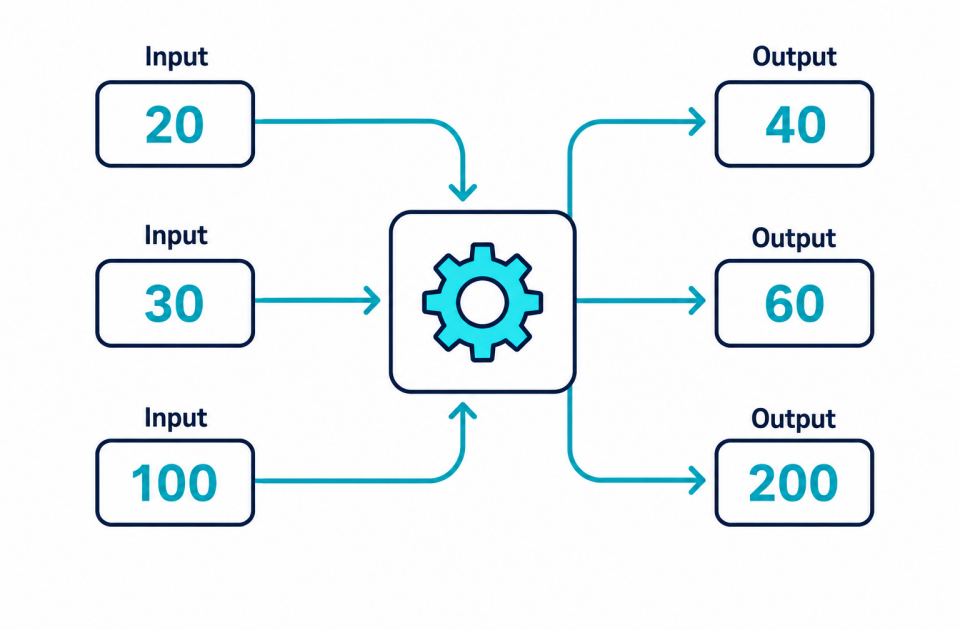
```
def convert_to_fahrenheit(celsius):
    fahrenheit = (celsius * 9/5) + 32
    return fahrenheit

temp1 = convert_to_fahrenheit(20)
temp2 = convert_to_fahrenheit(30)
```



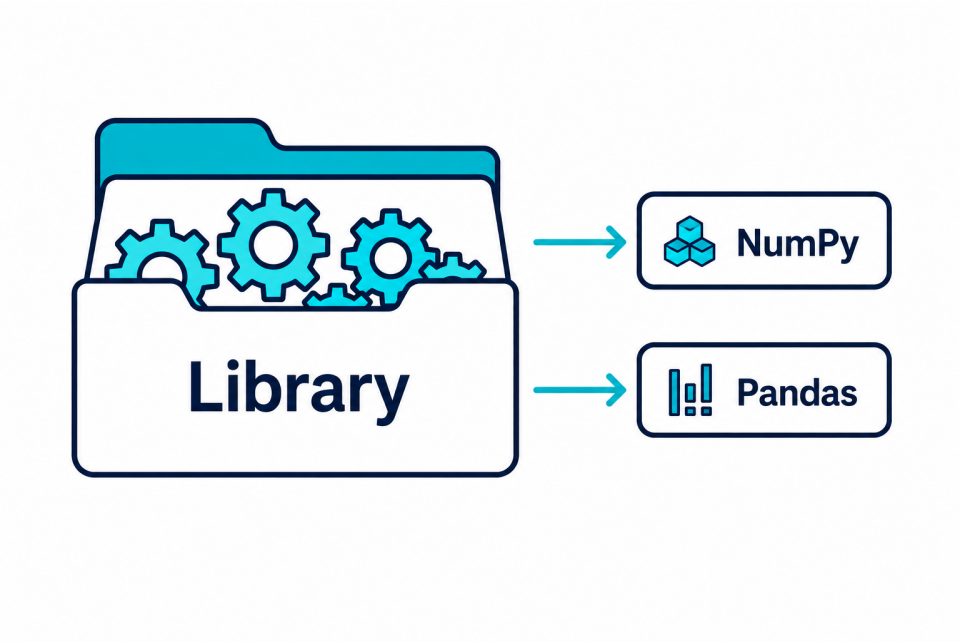
A function as a machine: input goes in, a result comes out.

The def keyword defines a new function. celsius here is called a parameter — a placeholder for whatever value gets passed in when the function is actually used. return sends a result back out of the function so it can be stored or used elsewhere. Once this function exists, you can call convert_to_fahrenheit() as many times as you want with different values, without rewriting the calculation each time.



The same function called with different inputs, producing a different output each time.

Functions matter enormously in AI work specifically, because most of what you'll do involves calling functions that other people have already written — a library like NumPy or a machine learning framework is really just a giant collection of pre-built, well-tested functions. Understanding how functions work, even simple ones you write yourself, is what makes those more powerful tools make sense later.



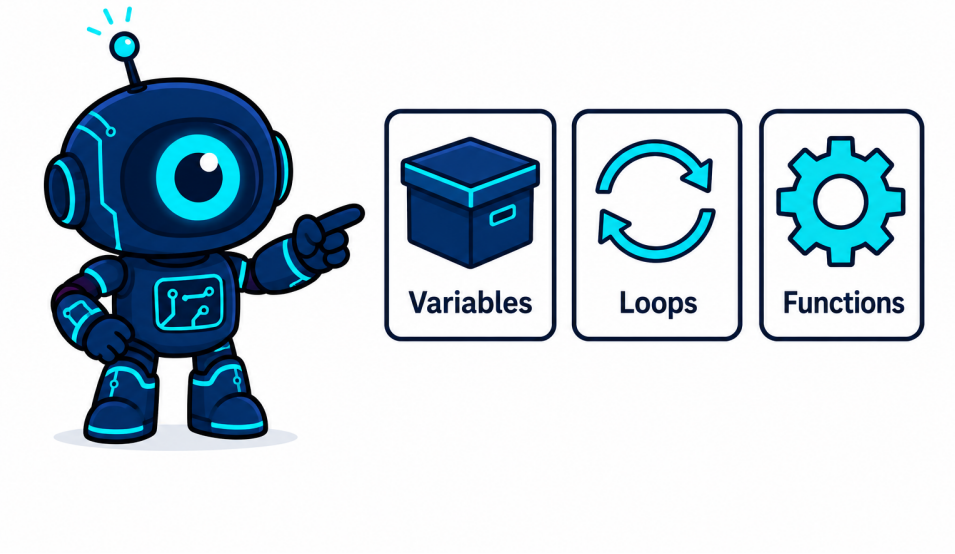
A library is a large collection of pre-built functions, ready to use.

Why this specific trio matters for AI

Every machine learning workflow you'll encounter follows the same underlying shape: load data (often using loops to process it), transform or clean it (frequently inside functions you write or call), and repeat calculations across large amounts of data (loops again, or functions that internally use loops). You won't be writing much raw looping code once you start using specialized libraries — they handle a lot of that efficiently behind the scenes — but understanding what's happening underneath is what lets you debug problems, understand error messages, and eventually customize behavior instead of just copying code you don't fully understand.

Where this leaves you

By the end of this lesson, you should be comfortable creating variables to hold different types of data, writing a simple for loop to process a list of values, and writing a basic function that takes an input and returns a result. That's the foundation the next lessons — NumPy and Pandas — are built directly on top of.



The three building blocks from this lesson: variables, loops, and functions.